DSTM Airport Simulation Communication Counts

Nicholas Cassarino

June 24, 2026

1 Introduction/Detour

If you are interested in an explanation of the simulation and our DSTM vs MPI versions, those details are located in (*/dstm/examples/airport_sim/scripts/scales*). This report is explicitly detailing the number of MPI communication calls used in both the DSTM and MPI versions of this simulation.

2 Experiment

We run a weak scaling experiment. We log the number of MPI communications used throughout the simulation. This logs were placed directly within the source code of both DSTM and the MPI airport simulation. Therefore, the *dstm_log()* calls may or may not be present within the github repository. In the DSTM example, these specific logs bloat our log files greatly.

Configuration.

- Airports per rank (fixed): 32
- Timesteps per run: 8
- Per-arrival synthetic work: 1,000 iterations
- Rank sweep: $P \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$
- Total airports (varies): $32 \cdot P$, from 32 at P=1 to 8192 at P=256
- Nodes (Cobalt allocation): 8
- Ranks-per-node: $\max(1, P/8)$, distributed via `--map-by ppr:PPN:node`

Builds compared.

- `dstm`: original DSTM-backed communication, at `../airports/airports`.
- `mpi`: faithful MPI port using `Isend`/`Irecv` with per-timestep barrier, at `../mpi_air/airports`.

3 Results

Here `Send_h` refers to sends within DSTM that occur from heartbeats. Likewise, `Send_b` refers to sends within DSTM that occur from barriers.

It is evident that communication counts are dominated by heartbeat sends and recvs within the DSTM version. While the MPI version scales more appropriately.

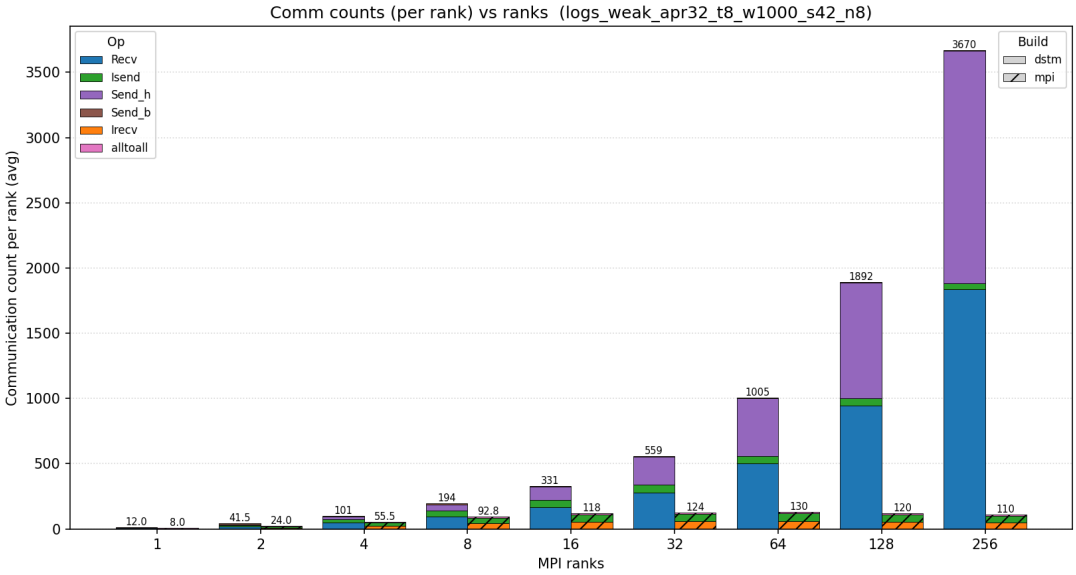


Figure 1: Weak scaling: per-rank MPI communication operations vs. rank count for the DSTM and MPI builds, with the per-rank airport count held fixed at 32. At each rank count the two builds are shown side-by-side, and each bar is stacked by operation type (`Recv`, `Irecv`, `Isend`, `Send`, `Send h`, `Send b`, `alltoall`). Lower bars indicate fewer communications issued to complete the same per-rank workload.